WORKSHOP 6 ชั่วโมง • HANDS-ON TRAINING

Mastering Local AI Stack 2026

การพัฒนาเวิร์กโฟลว์ AI สร้างข้อความและรูปภาพบนเครื่องคอมพิวเตอร์ส่วนบุคคล

เชื่อมระบบอัตโนมัติแบบ Offline 100% ไม่พึ่งพา Cloud API และไม่มีค่าใช้จ่ายรายเดือน

Jupyter Notebook
Python Orchestrator

LM Studio (:1234)
Local LLM Server (3B)

ComfyUI (:8188)
Node Diffusion API

Setup Guide: https://dhanabhon.github.io/mastering-local-ai-stack/



https://shor10.co/r2xdcjr

ทำไมต้อง Local AI Stack?

ข้อได้เปรียบของการรัน LLM และ Image Generation บนฮาร์ดแวร์ในเครื่องตัวเอง

1. Data Privacy 100%

- ข้อมูลเอกสารภายในและบทความภาษาไทยประมวลผลในเครื่องทั้งหมด
- ไม่ต้องส่งข้อมูลความลับขึ้น Cloud Server ภายนอก
- ทำงานได้แม้ไม่มีอินเทอร์เน็ต (Offline Mode)

2. Zero API Cost

- ไม่เสียค่า Token รายเดือนหรือค่าเช่า GPU บน Cloud
- ทดลองรัน Loop สกัดข้อมูลหรือเจนภาพซ้ำได้ไม่จำกัด
- ใช้ทรัพยากรเครื่องที่มีอยู่ให้คุ้มค่าสูงสุด

3. Full API Automation

- ไม่ได้ใช้แค่กดผ่านหน้าเว็บ GUI ทีละครั้ง
- เปิด Local Server คุยผ่าน REST API มาตรฐาน
- ใช้ Python ควบคุม Pipeline ตั้งแต่ต้นจนจบ

เป้าหมายวันนี้ : เปลี่ยนเครื่อง PC (VRAM 6GB) และ Mac (Unified Memory 16GB) ให้เป็น AI Server ส่วนตัว

ตารางกำหนดการอบรม (6-Hour Schedule)

เวลา	Module	หัวข้อการเรียนรู้และกิจกรรมปฏิบัติการ	ไฟล์ใบงาน (Lab)
09:00 - 10:15	**Module 1**	Jupyter Notebook Basics, การจัดการ JSON / Pandas DataFrame และเช็กสถานะ REST API ด้วย requests	01_python_and_api_basics.ipynb
10:30 - 12:00	**Module 2**	Local LLMs with LM Studio: โมเดล GGUF 3B, ตั้งค่าพารามิเตอร์ และสกัดข้อมูลบทความไทยเป็น JSON	02_local_llm_integration.ipynb
13:00 - 14:30	**Module 3**	Node-Based Image Generation with ComfyUI: โครงสร้าง SD 1.5 (512x512) และ Export API JSON	sd15_txt2img_api.json
14:45 - 16:00	**Module 4**	Capstone Integration: เชื่อม Jupyter -> LM Studio -> ComfyUI วาดภาพจากหัวข้อภาษาไทยอัตโนมัติ	04_end_to_end_ai_pipeline.ipynb

Hardware Target & การบริหาร VRAM 6GB

6 GB

เพดาน VRAM ของ RTX 3050
ต้องคุมงบหน่วยความจำไม่ให้เกิด OOM

~4.2 GB

VRAM รวมเมื่อรันพร้อมกัน
LLM 3B (~2.2GB) + SD 1.5 (~2.0GB)

เครื่องห้องปฏิบัติการ (Windows PC)

- CPU: Intel Core Ultra 7 265 (20C/20T, NPU 13 TOPS)
- GPU: NVIDIA GeForce RTX 3050 (6GB GDDR6)
- RAM: 16GB DDR5-5600MT/s | SSD 1TB PCIe Gen4
- แนวทาง: ใช้โมเดล 3B Q4_K_M + SD 1.5 (512x512)

เครื่องส่วนตัวผู้เรียน (macOS Apple Silicon)

- SoC: Apple M1 / M2 / M3 / M4
- Memory: Unified Memory 16GB ขึ้นไป
- แชร์หน่วยความจำร่วมกันระหว่าง CPU และ GPU (Apple Metal / MPS)
- รันทั้ง LM Studio และ ComfyUI พร้อมกันได้ลื่นไหล

Module 1: Jupyter Notebook & REST API

พื้นฐานสำคัญก่อนเชื่อมต่อโมเดล AI

- Interactive Computing:
  แยกหน้าที่ Markdown Cell (บันทึก) กับ Code Cell (รันคำสั่งด้วย Shift + Enter)
- Python Kernel & Virtual Env:
  ตรวจสอบว่า Kernel ชี้ไปที่ .venv ของโปรเจกต์
- File I/O ภาษาไทย:
  อ่านไฟล์ sample_article.txt ด้วย encoding="utf-8"
- JSON & Pandas DataFrame:
  แปลงข้อมูล Dict/List เป็น JSON Payload และแสดงตารางสรุปผลด้วย pd.DataFrame()

การเช็กสถานะ Local Server ด้วย requests

```
import requests

# 1. เช็กสถานะ LM Studio API (:1234)
res_llm = requests.get("http://localhost:1234/v1/models", timeout=3)
print(res_llm.status_code)  # 200 OK

# 2. เช็กสถานะ ComfyUI Server (:8188)
res_comfy = requests.get("http://127.0.0.1:8188/system_stats",
timeout=3)
print(res_comfy.json())
```

ใบงานที่ 1:
01-jupyter-basics/01_python_and_api_basics.ipynb
ฝึกอ่านบทความภาษาไทย แปลงตารางการใช้ VRAM และยิงเช็ก HTTP 200 ของทั้ง 2 เซิร์ฟเวอร์

Module 2: Local LLMs with LM Studio

GGUF & Quantization

- ฟอร์แมต GGUF ออกแบบมาให้รันบน GPU + CPU ได้ยืดหยุ่น
- บีบอัดน้ำหนักโมเดลระดับ Q4_K_M (4-bit Quantization)
- แนะนำ Qwen2.5-3B-Instruct หรือ Llama-3.2-3B-Instruct
- ใช้ VRAM เพียง ~2.2 GB

Context & GPU Offload

- Context Length: ตั้งไว้ที่ 2048 - 4096 tokens เพื่อประหยัด KV Cache บนการ์ดจอ
- GPU Offload: ส่งเลเยอร์โมเดลไปคำนวณบน GPU
- หาก VRAM ตึง ลดเหลือ 50-70% ให้ CPU ช่วยประมวลผลได้

OpenAI-Compatible API

- เปิดแถบ Developer -> กดปุ่ม Start Server ที่พอร์ต 1234
- Endpoint: http://localhost:1234/v1
- ใช้ไลบรารีมาตรฐาน from openai import OpenAI เรียกใช้งานได้ทันที โดยไม่ต้องใช้ API Key จริง

การปรับค่าพารามิเตอร์ตอนเรียก API:

- System Prompt: กำหนดบทบาทและบังคับโครงสร้างคำตอบ (เช่น สั่งให้ตอบเป็น JSON Array เท่านั้น)
- Temperature: ตั้งต่ำ (0.1 - 0.3) สำหรับงานสกัดข้อมูล JSON ที่ต้องการความแม่นยำ | ตั้งสูง (0.7) สำหรับงานแต่งคำบรรยายภาพ

Lab 2: สกัดข้อมูลบทความภาษาไทยเป็น JSON & Pandas

ขั้นตอนการทำ Structured Data Extraction

1. อ่านไฟล์บทความภาษาไทย
ดึงข้อความจาก ../assets/sample_article.txt

2. กำหนด System Prompt บังคับ JSON
สั่งให้ดึงประเด็นสำคัญ 4-5 ข้อในรูปแบบ JSON Array ที่มีคีย์:
• topic (หัวข้อประเด็นสำคัญ)
• category (Hardware / Model / Workflow / Benefit)
• key_takeaway (สรุปใจความสำคัญ 1 ประโยค)

3. คลีนข้อความและแปลงเป็นตาราง
ใช้ Regular Expression ตัด ```json ออก แล้วโหลดเข้า pd.DataFrame()

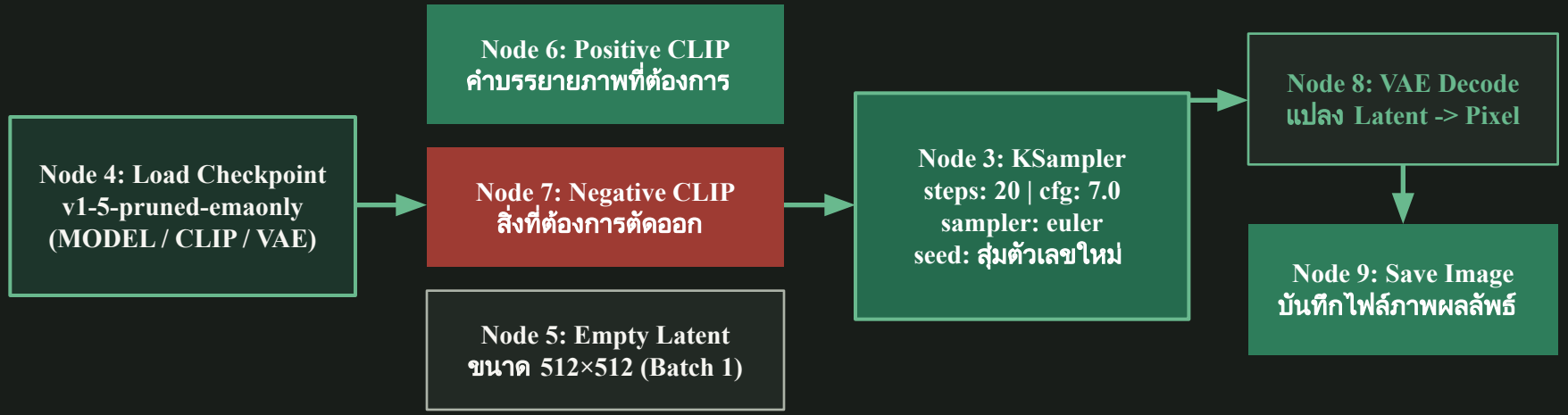
ตัวอย่างโค้ดใน 02_local_llm_integration.ipynb

```
from openai import OpenAI
import json, re, pandas as pd

client = OpenAI(base_url="http://localhost:1234/v1",
api_key="lm-studio")
res = client.chat.completions.create(
    model=loaded_model_id,
    messages=[
        {"role": "system", "content": sys_prompt},
        {"role": "user", "content": article_text}
    ],
    temperature=0.1, max_tokens=800
)
items = parse_json_safely(res.choices[0].message.content)
df_summary = pd.DataFrame(items)
display(df_summary)
```

Module 3: Node-Based Image Generation (ComfyUI)

โครงสร้างการไหลของข้อมูลใน Stable Diffusion 1.5 Workflow (512×512)



หน้าที่ของแต่ละองค์ประกอบหลัก:

- Checkpoint (SD 1.5): ไฟล์โมเดลหลัก (.safetensors ~2GB) ประกอบด้วย 3 ส่วนย่อยคือ UNet (MODEL), CLIP, และ VAE
- CLIP Text Encode: แปลงข้อความภาษาอังกฤษ (Positive & Negative Prompt) ให้เป็นเวกเตอร์ตัวเลขที่โมเดลเข้าใจ
- KSampler: หัวใจหลักในการลดสัญญาณรบกวน (Denoising) บนพื้นที่ Latent Space ตามจำนวน Steps (20) และค่า CFG (7.0)
- VAE Decode: แปลงข้อมูลจาก Latent Space ให้ออกมาเป็นรูปภาพพิกเซลขนาด 512×512 (ใช้เวลาเรนเดอร์ ~2-3 วินาที)

Lab 3: การส่งออก ComfyUI เป็น API Format JSON

วิธีเปิด Developer Mode เพื่อเซฟไฟล์ API

1. เปิดหน้าเว็บ ComfyUI ที่ http://127.0.0.1:8188
2. คลิกปุ่มรูปเฟือง (Settings) มุมขวาบน
3. เปิดสวิตช์ Enable Dev mode Options
4. กดปุ่ม Save (API Format) เพื่อได้ไฟล์ JSON สำหรับส่งเข้า REST API

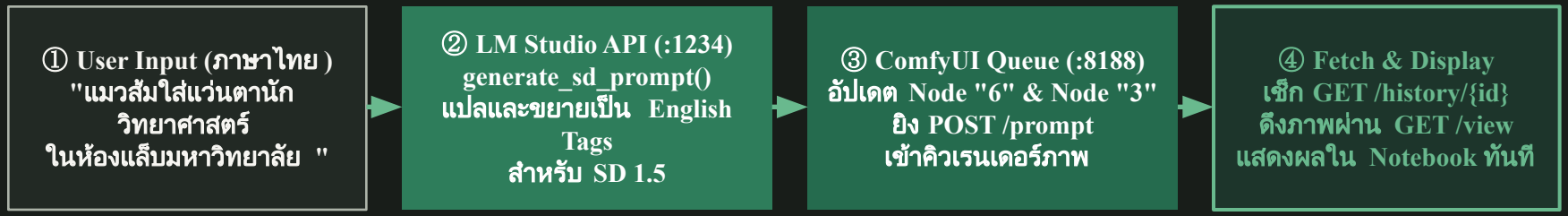
ข้อควรระวัง:
ไฟล์ Save ปกติเก็บตำแหน่งกล่องบนหน้าจอ GUI แต่ไฟล์ Save (API Format) เก็บเฉพาะโครงสร้าง Node ID และ inputs สำหรับยิงไปที่ POST /prompt

โครงสร้าง sd15_txt2img_api.json ที่แก้ไขด้วย Python

```
{
 "3": {
  "class_type": "KSampler",
  "inputs": {
   "seed": 156680208700286, // <- สุ่มค่าใหม่ทุกรอบ
   "steps": 20, "cfg": 7.0, "sampler_name": "euler"
  }
 },
 "6": {
  "class_type": "CLIPTextEncode",
  "inputs": {"text": "masterpiece, best quality..."} // <- แทนที่ด้วย
Prompt
 },
 "9": {
  "class_type": "SaveImage",
  "inputs": {"filename_prefix": "Workshop_Output"}
 }
```

Module 4: Full-Stack Local AI Pipeline (Capstone)

สถาปัตยกรรมการทำงานร่วมกันแบบ End-to-End ผ่าน Python Orchestrator




```
# โค้ดหลักใน 04_end_to_end_ai_pipeline.ipynb
positive_prompt = generate_sd_prompt(user_topic)            # 1. ให้ LM Studio แต่ง English Prompt
workflow = json.loads(WORKFLOW_JSON_PATH.read_text("utf-8")) # 2. โหลดเทมเพลต sd15_txt2img_api.json
workflow["6"]["inputs"]["text"] = positive_prompt           # 3. ใส่ Prompt ลงใน Node "6"
workflow["3"]["inputs"]["seed"] = random.randint(1, 10**14)  # 4. สุ่ม Seed ใหม่ใน Node "3"
prompt_id = queue_prompt(workflow)["prompt_id"]             # 5. ส่งเข้าคิว POST /prompt
result_image = wait_and_fetch_image(prompt_id)              # 6. รอรับภาพผ่าน /history และ /view
display(result_image)
```

เทคนิคบริหาร VRAM 6GB & การแก้ปัญหา (Troubleshooting)

เมื่อเปิด LM Studio คู่กับ ComfyUI บนการ์ดจอ 6GB

1. คุมขนาดโมเดลให้ไม่เกินงบ
• LM Studio: โมเดล 3B Q4_K_M (~2.2 GB)
• ComfyUI: SD 1.5 512×512 (~2.0 GB)
• ใช้ VRAM รวม ~4.2 - 4.5 GB (ไม่เกิน 6GB)

2. จำกัด Context Length ใน LM Studio
• ตั้งค่าไม่เกิน 2048 tokens เพื่อลดพื้นที่ KV Cache

3. แบ่งงานให้ CPU & ใช้แฟล็ก --lowvram
• ลดสไลเดอร์ GPU Offload ใน LM Studio ลงมาที่ 50%-70% ให้ Intel Core Ultra 7 ช่วยคำนวณ
• เติม --lowvram ในไฟล์รัน ComfyUI บน Windows

แนวทางแก้ปัญหาที่พบบ่อยระหว่างทำ Lab

• ConnectionRefusedError (:1234 หรือ :8188)
-> ตรวจสอบว่ากดปุ่ม Start Server ใน LM Studio และเปิดหน้าต่างรัน ComfyUI ทิ้งไว้แล้ว

• ชื่อไฟล์ Checkpoint ไม่ตรง (KeyError / HTTP 400)
-> เช็กชื่อไฟล์ในโฟลเดอร์ models/checkpoints/ หรือส่งพารามิเตอร์ custom_checkpoint เข้าฟังก์ชัน

• โมเดล LLM ตอบ JSON มีข้อความเกริ่นนำติดมา
-> ลดค่า temperature ลงเหลือ 0.1 และใช้ฟังก์ชัน parse_json_safely() ดึงเฉพาะส่วน [...]

พร้อมลงมือทำ Workshop!

ขั้นตอนเริ่มต้นใช้งานคลังใบงาน (Quick Start)

```
git clone https://github.com/Dhanabhon/mastering-local-ai-stack.git
cd mastering-local-ai-stack
python -m venv .venv
# Windows: .venv\Scripts\activate  |  macOS: source .venv/bin/activate
pip install -r requirements.txt
jupyter lab
```

1. เปิด LM Studio Server ที่พอร์ต 1234 | 2. เปิด ComfyUI ที่พอร์ต 8188 | 3. รันใบงาน 01 ถึง 04 ตามลำดับ

คู่มือการติดตั้งและเตรียมเครื่องออนไลน์: https://dhanabhon.github.io/mastering-local-ai-stack/